SPADES.IND

User and Reference Manual

Version 0.1.8 | English edition

Synthetic individual data through Bayesian-network proposals, energy-driven flight and simulated-annealing landing.

Classification, continuous-response regression and fidelity-only generation.

Pierpaolo Massoli

31 October 2026

This manual documents the supplied implementation. It combines operating instructions, workflow explanations and the function reference. The case-study input files retain their original train/holdout split.

Contents

1. Installation and first use

2. Generation workflow and three response scenarios

3. Controls, parallel execution and saved state

4. Fidelity, privacy and external utility

5. Classification paper case study

6. Troubleshooting and interpretation

7. Function reference

Use the named chapters and function headings to navigate. Code examples require the named input objects and packages; short examples illustrate the interface rather than a production-size experiment.

1. Installation and first use

Install from source

SPADES.IND includes C++ code. Source installation needs an R-compatible compiler toolchain (Rtools on Windows). Install Rcpp, withr and rngtools, and the modelling dependencies used by the intended workflow: ranger, xgboost, bnlearn, Matrix and rpart. Building HTML vignettes additionally requires knitr, rmarkdown and Pandoc.

```
install.packages("path/to/SPADES.IND", repos = NULL, type = "source")
library(SPADES.IND)
help("spades_synthesise", package = "SPADES.IND")
vignette("workflow", package = "SPADES.IND")
```

Inputs and outputs

Use a data frame with named columns, complete supported types and at least two nonconstant variables for model fitting. Keep a real holdout outside generation. The main result includes data (released values), categorical (selected bins/categories), prepared, model, pool, landing, filtering and response_protocol. Save the result if its models, selected indices and discretizers are needed later.

```
saveRDS(result, "path/to/result.rds")
write.csv(result$data, "path/to/synthetic_data.csv", row.names = FALSE)
result$landing$summary
result$landing$components
```

The CSV contains the release; it does not preserve all provenance needed to resume flight or reconstruct an analysis. Use short persistent output paths on Windows. Do not overwrite a previous analysis with incompatible settings.

2. Generation workflow and three response scenarios

Scope and data preparation

SPADES.IND generates a categorical candidate pool (flight), selects a fixed-size subset (landing), and reconstructs numerical variables within their original intervals. Supply training data only. Reserve a real holdout before preparation, binning, model fitting or guide construction.

Rows with actual missing values, textual `NA`, internal missing markers or additional user-declared `missing_tokens` are omitted once. The same cleaned reference is used throughout. Missingness is not modelled or generated. A literal `Missing` category is ordinary data unless explicitly declared missing. Constant columns and unsupported column types require preprocessing.

Examples below are opt-in and use artificial data. They are not executed during vignette building; the corresponding small workflows are tested in the package test suite. Their very short chains demonstrate calls, not statistical quality.

Three complete workflows

The default main workflow uses six chains and six workers. The small example uses one worker and intentionally tiny settings. Production settings require validation for the application; there is no universal optimal schedule.

```
# Run explicitly after installing SPADES.IND. Uses artificial data only.

local({
  if (!requireNamespace("SPADES.IND", quietly = TRUE)) stop("Install SPADES.IND first.")
  withr::local_seed(444L)
  d <- data.frame(x = rnorm(120), group = rep(letters[1:8], 15),
                  category = rep(LETTERS[1:6], 20))
  d$income <- 10 + 3*d$x + as.integer(factor(d$group)) + rnorm(120)
  small_run <- function(data, target, mode) SPADES.IND::spades_synthesise(
    data, n=8L, target=target, mode=mode, n_pool=20L, n_bins=4L,
    chains=1L, workers=1L, seed=84L, pairs=list(), conditional_on="group",
    fit_control=list(n_fake=30L,nrounds=3L,bn_restart=0L,nthread=1L),
    flight_control=list(burnin=3L,thin=1L,log_every=20L),
    landing_control=list(L=30L,pilot=10L,log_every=30L))
  categorical <- d
  categorical$income <- factor(ifelse(d$income > median(d$income), "high", "low"))
  a <- small_run(categorical, "income", "classification")
  b <- small_run(d, "income", "regression")
  c <- small_run(d, NULL, "none")
  print(lapply(list(classification=a, regression=b, no_response=c), function(z) head(z$data)))
  # These tiny settings demonstrate API use; they do not establish statistical quality.
})
```

Use `mode="classification"` explicitly for numeric class codes. With `mode="auto"`, a numeric response implies regression. With `target=NULL, mode="none"`, there is no response guide or response-utility objective. RF-based association selection, the BN and the XGB plausibility discriminator still serve their other roles. `pairs=NULL` selects pairs automatically; `pairs=list()` intentionally requests none.

What flight optimizes

For a fixed chain archive A, let q be the fitted BN mass, p the XGB real-class score, and d the normalized nearest-record Hamming distance. The implemented energy has the form

H_A(x) = -log q(x) + lambda_P (1-p(x)) + lambda_R exp(-d(x,R)/tau_R) + lambda_D exp(-d(x,A)/tau_D).

An empty archive contributes zero diversity energy. BN independence proposals cancel the BN term in the Hastings ratio. Both old and proposed states are compared against the same archive. The archive grows during collection and is local to a chain, so this is adaptive sampling, not a claim of draws from one fixed stationary distribution. Duplicate synthetic contents are retained. Exact categorical reference matches are excluded. These

mechanisms are not formal anonymity or differential-privacy guarantees.

The staged API separates preparation, model fitting, flight and landing. Use `spades_resume_flight()` for checkpointed flight recovery; consult its help for the checkpoint arguments. Reusing the fitted model avoids refitting it. Calling the main workflow again is not itself a model-reuse operation.

Landing objective and fidelity

For k variables and B distinct selected pairs, the default component count is J = k + B + (k - 1). Fidelity is the arithmetic mean of these J components:

F(T) = mean of the J component discrepancies; H(T) = F(T) + U(T).

Each marginal or joint-pair component is total variation distance. Each conditional component averages within-stratum TVD with the reference stratum proportions. A reference stratum absent from the target receives discrepancy one. The highest-cardinality prepared variable conditions the distributions unless `conditional_on` specifies another column. This formulation is a mean across components, not a maximum and not a mean giving equal weight to component families. RF importance selects associations; it does not set their energy weights. There is no landing diversity penalty in this workflow.

Classification utility is the absolute difference between the fixed RF guide's accuracy on the selected target and its accuracy on the training reference. For no response, U is zero. Selection swaps one pool index at a time, with unique selected indices but possibly duplicated record contents. The reported result is the final SA state. An optimum is not certified.

Continuous response: categorical fidelity, numerical utility

The cleaned numerical response is partitioned using type-7 quantiles, requesting ten bins by default. Repeated boundaries are merged; ten nonempty bins are not guaranteed. The response protocol reports effective bins and counts.

The pool remains categorical, including the response bin. A fixed RF regression guide is fitted on categorical predictors and the real numerical response. For each candidate swap, a fresh response value is drawn uniformly inside the candidate bin. Integer-valued input uses the admissible integers in that interval. The objective adds

U(T) = abs(E(T) - E(R)) / s_R.

where E is RMSE or MAE and s_R is the real response standard deviation. Accepting a proposal retains both its pool index and response value. Rejecting it preserves both old values. Final accepted responses are never redrawn. Numerical predictors are reconstructed after landing, just as in classification.

The proposal is interpreted relative to the product of within-bin uniform base measures and uniform index selection. The within-bin proposal ratio cancels the base-measure ratio. Unequal-width bins do not imply symmetry with respect to ordinary numerical Lebesgue measure.

The low-level `spades_utility(mode="regression")` helper remains a fixed-numerical- pool interface for compatibility. It is not the dynamic regression branch of `spades_synthesise()`. Likewise, fixed-score regression in `spades_target_stability()` is distinct. Dynamic-response repetitions should use `spades_land_response()` with explicit seeds, preserving each accepted response. Interrupted dynamic SA is not currently resumed from an intermediate state. Uniform reconstruction is not an additional proven privacy guarantee.

Evaluation after generation

See `vignette("evaluation", package="SPADES.IND")` for fidelity, privacy, membership inference, fixed-pool stability and external utility. The classification paper protocol is described in `vignette("paper-case-study", package="SPADES.IND")`.

Supported interfaces and research modules

The supported workflow comprises preparation, automatic fidelity structure, model fitting, flight/resume, landing, reconstruction, fidelity/privacy audits, fixed-pool stability and external utility. Historical research-module accessors are retained for compatibility and helper dependencies. Their presence does not establish validation of every historical conformal, sensitivity or target-size experiment.

3. Controls, parallel execution and saved state

Main and model defaults

The main function defaults to n_pool = max(1000, 12*n), n_bins = 20, target_bins = 10, chains = 6, workers = 6 and seed = 4123. The pool size is a requested count, not a guarantee of a desired fidelity optimum. Model-fitting defaults are n_fake = 15000, nrounds = 900, bn_restart = 50 and nthread = 1. Override these through fit_control in the main workflow. High-cardinality data and small finite domains may require adapting the number of negative records. Model parameters and seed changes define a new analysis. ## Flight controls

The supported control names and defaults are: - burnin = 1000: transitions before retained collection. - thin = 5: transitions per retained record. - log_every = 10000: progress interval in transitions per chain. - cache_size = 4096: static candidate-score cache capacity. - lambda_plausibility = 1; lambda_privacy = 1; lambda_diversity = 0.7. - tau_privacy = 0.25; tau_diversity = 0.25. A chain assigned m retained records runs burnin + thin*m transitions. These controls are fixed for a checkpointed run. They do not constitute universally optimal tuning values. ## Landing controls

L = 1000000 sets proposal iterations, pilot = 400 sets the temperature pilot length, and log_every = 10000 controls progress. The pilot calibrates temperature; it does not change the initial selected state. The implemented schedule and one-record swap kernel are internal to the landing implementation. Unknown control names are rejected. ## Parallelism

Flight uses parallel::makePSOCKcluster and parallel::parLapply. The current workflow does not require registering a doParallel backend. Logical chains define independent archives and saved RNG state; workers execute those chains. Worker count is capped by chain count. RF operations support num.threads; XGB operations support nthread. fit_control$nthread controls fitting separately from the main workers parameter. External utility uses its threads argument for XT and XGB; bagged CART builds its trees sequentially. Landing is a sequential SA chain with compiled incremental updates. Avoid oversubscribing CPUs when running several independent analyses. Six workers is a configurable default, not a requirement.

Use a new directory for each planned flight. `model` is a fitted object returned by spades_fit; train it once on the user's training data. No case-study names, paths or saved historical models are required by these APIs.

```
library(SPADES.IND)
pool <- spades_flight(
  model, n_pool = 110000L, chains = 6L, workers = 6L, seed = 123L,
  checkpoint_dir = "path/to/flight_run_01",
  checkpoint_every = 10000L,
  control = list(log_every = 10000L)
)
```

After an interruption, including a new R session, use only:

```
library(SPADES.IND)
pool <- spades_resume_flight("path/to/flight_run_01", workers = 6L)
```

The original fitted model is loaded from request.rds; no call to spades_fit or spades_synthesise is required. A completed run returns its saved pool immediately. Checkpoint interval and worker count may change; logical chains, seeds, model, requested total size, energy controls and software environment must stay fixed. This is continuation to the original total, not extension to a larger pool. Existing 0.1.0 runs cannot be resumed retroactively without these checkpoints.

Checkpoint_every counts MH iterations PER CHAIN, including burn-in. Each chain saves at initialization, every interval, and completion. At an abrupt interruption, at most the iterations since its last checkpoint must be replayed. This does not repeat training or the completed portion. Archive, current state, energy scores, static-score cache, counters and RNG are saved together. Initialization before its first successful checkpoint may need to be repeated.

Two committed generations are retained per chain. A temporary interrupted write is ignored. A corrupt latest generation is quarantined and the previous one is used. If neither is valid, execution stops rather than silently restarting. Checkpoint writes use immutable filenames and rename on the same filesystem; this is not a guarantee against disk failure or loss of unflushed OS buffers. Use a local persistent directory, with enough space for model and chain archives.

A RUNNING.lock directory prevents two callers writing the same run. Normal exit or an R error releases it after stopping workers. After a process/PC crash it may remain. FIRST stop any surviving workers of that run, THEN remove only:

```
unlink("path/to/flight_run_01/RUNNING.lock", recursive = TRUE)
```

Do not remove a live run's lock. Checkpoint files and request.rds must be retained.

Controlled pause

```
paused <- spades_flight(model, 110000L, chains=6L, workers=6L,
  checkpoint_dir="path/to/flight_run_01", run_for=5000L)
# Continue later with spades_resume_flight(...).
```

run_for is the maximum additional transitions per chain in this invocation. A paused call returns spades_flight_paused, not a pool suitable for landing. Different chains may have reached different checkpoints after an abrupt stop. Each resumes from its own saved state; completed chains require no transitions.

Scope

The one-call spades_synthesise workflow still fits fresh models when invoked. For long restartable work, use the staged prepare/structure/fit/flight/land API and resume the flight with spades_resume_flight. Save RF structure and utility objects with saveRDS if needed for later stages. No new fitting is triggered by spades_resume_flight. This release does not implement landing checkpoints.

Exact continuation requires the same R, SPADES.IND, XGB, BN and Matrix versions and numerical platform. RNG kind is restored from the request. Native XGB model bytes are stored, avoiding serialization of external pointers.

4. Fidelity, privacy and external utility

Evaluation inputs

Assess the final released data, not an obsolete reconstruction. Keep the cleaned training reference, categorical target, saved discretizers, selected pairs and conditioning variable with each release. The real nonmember holdout must not have been used in generation or guide fitting. The calls below are templates for existing objects; they are not executed while building this vignette.

```
# result: a completed spades_synthesise() result

fidelity <- SPADES.IND::spades_fidelity_audit(
  reference = result$prepared$data,
  categorical = result$categorical,
  released = result$data,
  reference_values = result$prepared$original,
  discretizers = result$prepared$numeric_discretizers,
  pairs = result$landing$pairs,
  conditional_on = result$landing$conditional_on)
```

Reconstruction and fidelity checks

Always evaluate the final release as well as its categorical representation. Reapplying the saved discretizers must recover every selected bin. The package rejects bin crossings. Categorical fidelity is therefore invariant under valid reconstruction, but numerical distributions must still be assessed separately. Use `spades_fidelity_audit()` with the saved pairs, conditioning variable, discretizers, categorical target and final numerical release. It reports marginal, bivariate and conditional components, along with numerical distribution checks. Do not infer full joint-distribution fidelity from small selected-component errors.

Privacy and membership inference

`spades_privacy_audit()` uses the actual final release, training members and real nonmembers. It reports duplicates and exact matches and evaluates distance, density, nearest-neighbour and RF attacks. Candidate profiles shared between member and nonmember groups are excluded; candidate classes are balanced. Attack training, calibration and test partitions are separate. Calibration sets select thresholds and attacks; held-out attack test sets measure performance. Controls include permuted membership labels and a hypothetical copied release.

`spades_mia_stability()` repeats attack partitions on the same cached release and candidate features. Repeated splits overlap, so their variability is descriptive, not independent replication. Near-chance attack performance is limited empirical evidence for the tested attacks, not a privacy certificate. Candidate-source mismatch can confound the results. Audit outputs may contain record-level links; choose an explicit output directory outside the package source.

Stability and external predictive utility

Fixed-pool target stability repeats landing and reconstruction with different target seeds while preserving the pool and guide. It does not measure flight variability. For external utility use `spades_external_utility()` with a named list of final synthetic targets, the cleaned real training data and the same untouched real holdout. Use one fixed model seed and the same hyperparameters across targets. Ten targets with XT, bootstrap CART and XGB entail thirty synthetic fits plus three real baselines; no model-seed grid is required.

Classification reports accuracy and probabilistic metrics; regression reports RMSE, MAE and R-squared. Guide performance used by landing is not a substitute for this external assessment. Shared-holdout comparisons and repeated targets are dependent. Cache-compatible completed external fits can be reused.

Membership audit template

Use matching columns and complete member/nonmember data. The audit requires at least 200 eligible distinct profiles per membership class after exclusions; the tiny workflow examples are intentionally too small for this analysis.

```
privacy <- SPADES.IND::spades_privacy_audit(
  members = result$prepared$original,
  nonmembers = real_holdout,
  released = result$data,
  out_dir = file.path(audit_directory, "privacy"),
  seeds = 20261100L + seq_len(20L), threads = 6L)
```

External utility template

A named list permits comparison of multiple final targets. Keep the model seed fixed across the list so that varying the target is the intended experimental factor. Use `mode="regression"` for a numerical response. This is distinct from optimizing the fixed guide during landing.

```
utility <- SPADES.IND::spades_external_utility(
  reference = real_training,
  synthetic = released_targets,
  holdout = real_holdout,
  target = response_name,
  mode = "classification",
  models = c("XT", "Bagging_CART", "XGB"),
  seed = 20261026L, threads = 6L,
  out_dir = file.path(audit_directory, "external_utility"))
```

Reporting results

Report the number of cleaned training rows and excluded rows, target and pool sizes, fidelity components rather than only their mean, numerical reconstruction checks, duplicates, exact matches and MIA attack performance with controls. For multiple targets report the range and variability across targets, and retain per-target results. Report probabilistic classification metrics as well as accuracy. Distinguish uncertainty across targets from uncertainty across model seeds; the fixed-seed protocol does not estimate the latter.

Keep protocol settings, software versions, seeds and cached result provenance. Do not describe an incomplete or interrupted analysis as a completed evaluation. Near-chance membership attacks do not rule out untested attacks. Improved accuracy does not imply improved probabilistic predictions or full fidelity.

5. Classification paper case study

Scope

This vignette describes the classification case-study protocol: generation, target selection, numerical reconstruction and evaluation. Building this vignette does not execute the case-study experiments.

Reference and holdout

The study uses 9,000 real training records and a separate 1,000-record real holdout. Diagnosis is the categorical response. The eight variables are age, gender, civil status, occupation, physical activity, genetic predisposition, municipality of residence and diagnosis. Age is discretized for generation and fidelity and reconstructed within its original saved bin after landing. Municipality of residence is the highest-cardinality conditioning variable.

The holdout is not used to fit the BN, the plausibility discriminator, the association-selection forests or the landing utility guide. Final external predictive comparisons use this same real holdout.

Approved flight and landing

The approved flight uses six chains, BN proposals and plausibility, privacy and chain-local diversity energy terms. Duplicate synthetic record contents are retained. The approved landing selects 9,000 pool indices using one million single-record swap proposals. Its objective is mean categorical fidelity plus absolute fixed-RF accuracy gap, with coefficient one for each term. There is no additional diversity term in this landing.

The fidelity components include all eight marginals, seven distributions conditioned on municipality, and these five bivariate distributions:

- age and civil status;

- age and occupation;

- age and genetic predisposition;

- diagnosis and physical activity;

- genetic predisposition and diagnosis.

The twenty components receive equal weight in their arithmetic mean. Conditional discrepancies use real municipality proportions. This is the approved protocol, not earlier maximum-based or adaptive-weight variants.

Final numerical release

Only the corrected age reconstruction is used for final audits: applying the saved discretizer to reconstructed ages must reproduce every selected age bin. The selected categorical records are unchanged by this correction. Older releases with age-bin crossings must not be mixed with the corrected outputs. Final-release duplicates and exact matches are measured after reconstruction; duplicates are reported, not removed.

Fidelity, privacy and stability

Fidelity is assessed on selected categorical records and on the final numerical release. Privacy checks include exact matches, duplicates, nearest-neighbour features and membership attacks with separate attack training, calibration and test sets. Repeated membership splits reuse the same release and candidate features; they are not independent synthetic-data replications.

The target stability experiment generates ten targets from the same fixed pool, with different landing seeds and valid numerical reconstruction. It measures variation due to target selection and reconstruction, not variation in flight. The existing completed results are retained; documentation work does not require running these

experiments again.

External utility

XT, bagged CART and XGB are fitted to each of the ten final targets with one fixed model seed and the same model configuration. Each model is also fitted once to the real training reference. All fits are evaluated on the same 1,000 real holdout records: thirty synthetic fits and three real baselines.

Accuracy and probabilistic metrics answer different questions and must be reported separately. Repeated targets evaluated against a shared holdout do not provide independent test datasets. The case study validates a classification application; it does not provide empirical validation of the continuous-response extension.

Reproducibility

Reproduction uses the original training/holdout split, saved discretizers, model settings, seeds and software versions. Final audits use the corrected numerical release and the approved categorical selection. Preserve these inputs and the saved result tables together. A new generalized workflow call need not be bitwise identical to an earlier script run under another software environment.

Included inputs

The original CSV files are installed under `extdata/paper`. Their checksums and observed variable inventory accompany them. Read all columns as character first and convert age to integer; the diagnosis codes represent classes.

```
folder <- system.file("extdata", "paper", package="SPADES.IND")
training <- read.csv(file.path(folder, "training_data.csv"),
                     colClasses="character", na.strings=NULL)
holdout <- read.csv(file.path(folder, "test_real_1000.csv"),
                    colClasses="character", na.strings=NULL)
training$age <- as.integer(training$age)
holdout$age <- as.integer(holdout$age)
stopifnot(nrow(training)==9000L, nrow(holdout)==1000L)
```

The supplied training CSV includes literal `Missing` categories in physical activity and genetic predisposition. Preserve the original files for provenance. If these labels are declared missing when preparing an analysis, complete-case filtering changes the reference and must be reported explicitly. The installed `extdata/paper/README.md` describes the input conventions. Historical fitted models and approved result tables are separate from these input files.

Classification case-study input data

`training_data.csv`: 9,000 records used as the real training reference. `test_real_1000.csv`: 1,000 records reserved for evaluation. Both files have the same eight columns in the original order. Their bytes and original SHA-256 checksums are retained unchanged from the supplied archive. `data_inventory.csv` records observed cardinalities and literal missing markers.

The split is supplied, not recreated by a new random split. Duplicate profiles are retained. There are 7 distinct complete profiles shared by the files; profile equality does not establish that these are the same sampled individual.

The text `Missing` occurs in physical_activity and genetic_predisposition in the training file. This archive preserves the historical input. The package treats this literal label as a category unless included in `missing_tokens`. Declaring it missing invokes complete-case exclusion and changes the training reference; that is a different analysis from preserving this supplied file unchanged.

Read categorical codes as character values or factors, including diagnosis, occupation, physical_activity and genetic_predisposition. Age is numeric. Do not allow numeric diagnosis codes to select regression automatically. The loader in `examples/load_paper_data.R` returns typed data without fitting, generation,

filtering, deduplication or changes to the split.

Column descriptions (code meanings are not inferred):

Column / Representation / Description

municipality_residence / categorical / Municipality of residence

age / integer / Age; observed values 18 to 65

civil_status / categorical / Civil-status labels as supplied

gender / categorical / Gender labels as supplied

occupation / categorical / Codes 0 and 1

physical_activity / categorical / Codes 1 to 5 and literal Missing

genetic_predisposition / categorical / Codes 1 to 8 and literal Missing

diagnosis / categorical response / Codes 1 to 4

These files were supplied by the maintainer as the inputs used for the paper. The maintainer has indicated that they may be made public. No external source, collection method, codebook or data reuse license is inferred from the CSVs.

6. Troubleshooting and interpretation

Input and category errors

Unknown pool categories indicate that the pool and reference domains differ. Reuse the original prepared reference and saved discretizers. Constant columns are unsupported by model fitting. Numerical class codes require explicit classification mode or categorical typing. Actual missing values and configured textual tokens cause row exclusion during preparation; inspect filtering before fitting. ## Missing versus a category named Missing

The default textual token is NA. A literal Missing label is retained unless declared in missing_tokens. For the supplied paper data, changing this policy changes the reference. Do not silently replace a historical input when reproducing saved results. ## Interrupted runs

Resume a flight through spades_resume_flight rather than calling the main workflow again. Keep the original request and checkpoint files. If a stale lock exists, first confirm that the original process and workers have stopped. Dynamic landing has no intermediate-SA resume. Target-study caching reuses completed targets; an unfinished target landing must restart. ## Utility interpretation

Matching fixed-guide accuracy or error does not establish external predictive equivalence. Always train the evaluation models on final released numerical data and test on the real holdout. Log loss and Brier can worsen even when classification accuracy is preserved. Do not infer calibration alone from those metrics without additional analysis. ## Fidelity and privacy interpretation

A small mean fidelity discrepancy can coexist with a large individual component. Review the component table and numerical checks. No exact matches and near-chance membership attacks are descriptive observations, not a formal privacy guarantee. Preserve duplicates when evaluating the released file. ## Reporting a problem

Record sessionInfo(), SPADES.IND version, call, seeds, controls, worker and chain counts, and whether execution was fresh, resumed or cached. Supply a small artificial reproducible example and the complete error. Keep record-level candidate links and model caches out of an issue report.

7. Function reference

Signatures and parameter descriptions below are transcribed from the package help files. The main workflow, preparation, dynamic response landing, resume, audits and external utility have dedicated reference sections. Lower-level flight and landing controls are also described in Chapter 3.

Generate Synthetic Data With an Optional Response

Description

Prepare a complete-case reference, fit generation models, generate a categorical pool, select a target by simulated annealing, and reconstruct numerical values.

```
spades_synthesise(data, n, target = NULL,
  mode = c("auto", "classification", "regression", "none"),
  n_pool = max(1000L, 12L * n), n_bins = 20L,
  chains = 6L, workers = 6L, seed = 4123L,
  pairs = NULL, conditional_on = NULL,
  fit_control = list(), flight_control = list(), landing_control = list(),
  cache_dir = NULL, verbose = TRUE, missing_tokens = "NA",
  target_bins = 10L, regression_metric = c("RMSE", "MAE"))
```

Arguments

data

Training data frame only. Do not include the external holdout. Supported columns and complete-case filtering are described in spades_prepare.

n

Positive integer number of synthetic rows, no larger than the pool.

target

Name of the response variable, or NULL when there is no response.

mode

With "auto", a numeric response selects regression, any other response selects classification, and a missing target selects "none". Specify classification explicitly for numerical class codes.

n_pool

Requested number of pool rows. Duplicate row contents are retained.

n_bins

Requested number of bins for numerical variables other than the regression response.

chains

Number of logical flight chains.

workers

Flight worker count; also used for structure and utility random-forest threads.

seed

Nonnegative integer master seed, with room for workflow offsets. Caller RNG state is preserved.

pairs

List of two-column character vectors defining bivariate fidelity components. NULL requests automatic RF-based selection; list() disables these components.

conditional_on

Single conditioning column. If NULL, use the highest-cardinality prepared column; column order resolves ties.

fit_control

Named model-fitting overrides: n_fake, nrounds, bn_restart, bn_blacklist, and nthread.

flight_control

Named flight controls accepted by spades_flight, including burnin, thin, and log_every. See the workflow vignette and flight documentation.

landing_control

Named overrides: L (default 1000000), pilot (400), and log_every (10000). Each must be a positive integer.

cache_dir

Optional flight cache directory. For explicit interruption and checkpoint control use the staged flight API; see spades_resume_flight. The main function fits generation models before calling flight.

verbose

Print progress.

missing_tokens

Text labels treated as missing in addition to actual missing values and internal markers. Affected rows are removed before any fitting.

target_bins

Requested quantile bins for a regression response (default ten). Coincident boundaries are merged.

regression_metric

Regression guide error, RMSE or MAE.

Details

All modes use BN proposals with plausibility, distance-based privacy and diversity penalties in flight. Exact categorical reference matches are rejected; duplicate synthetic contents are retained. Diversity is evaluated against a chain-local archive, which makes flight adaptive.

Landing fidelity is the arithmetic mean of individual marginal, selected-pair and conditional TVD components. Conditional TVD is weighted by reference stratum proportions; an absent target stratum has discrepancy one. This is neither a maximum nor an equal-weight mean of three component families. Landing does not add a diversity penalty.

Classification adds the absolute accuracy gap from a fixed RF guide. Regression adds the absolute RMSE or MAE gap divided by the reference response standard deviation. Guide reference performance is measured on its training rows and is not external predictive validation. With no response, utility is zero.

For regression the pool and fidelity remain categorical. A guide uses categorical predictors and real numerical responses. Each proposal samples a numerical response uniformly inside its proposed bin (uniform over

admissible integers for integer-valued responses). Acceptance retains that value; rejection retains the previous state. Final response values are not sampled again. Other numerical columns are reconstructed after landing, respecting their original bins. See spades_land_response.

Selection is without replacement in pool indices, not in distinct row contents. The returned target is the final SA state. Neither globally optimal fidelity nor a formal privacy guarantee is asserted.

Value

A list containing data (released values), categorical (selected categorical records), landing, pool, prepared, model, mode, seed, filtering, and response_protocol.

Seealso

spades_prepare, spades_land_response, spades_fidelity_audit, spades_privacy_audit, spades_external_utility.

Prepare a common complete-case reference

Description

Remove records with missing values before binning or any model fitting. Convert supported columns to the categorical representation used by the workflow.

```
spades_prepare(data, nominal = NULL, ordinal = NULL, numeric = NULL,
               n_bins = 20L, method = c("quantile", "equal_width"),
               missing_tokens = "NA", verbose = TRUE)
```

Arguments

data

Nonempty data frame of supported raw columns.

nominal, ordinal, numeric

Optional column names partitioning the data. Defaults infer numeric columns and ordered factors, treating the remainder as nominal.

n_bins

Integer at least two, requested numeric bin count.

method

Numeric binning method.

missing_tokens

Additional textual missing labels. Actual NA/NaN and textual NA and __MISSING__ are always removed. Matching is case-sensitive after whitespace trimming.

verbose

Print the input, excluded and used record counts.

Details

A single row mask is applied before fitting numeric bins. The cleaned original and categorical frames are row aligned. Literal Missing is a normal category unless declared in missing_tokens. The downstream workflow uses this same reference and does not generate synthetic missingness.

Value

A spades_prepared list containing data, original, retained_rows, filtering, missing_policy, schema, numeric discretizers and controls.

Landing with a Continuous Response Drawn per Proposal

Description

Select categorical pool rows while sampling a continuous response within its categorical interval at each proposal. The response is retained on acceptance.

```
spades_land_response(pool, prepared, n, target,
  metric = c("RMSE", "MAE"), seed = 1122L, rf_seed = 1011L,
  threads = 1L, control = list(), pairs = list(),
  conditional_on = NULL, keep_trace = TRUE, verbose = TRUE,
  initial_indices = NULL, initial_y = NULL)
```

Arguments

pool

Categorical data.frame or spades_flight object.

prepared

Prepared real training data with a numeric target and quantile discretizer. The main workflow defaults to ten response quantile bins; ties merge boundaries.

n

Number of records selected without replacement by pool index. Duplicate row contents are retained.

target

Numeric response column name.

metric

RMSE or MAE, computed with a fixed regression forest guide.

seed

Landing seed, allowing two subsequent seed offsets.

rf_seed

Regression forest seed.

threads

Guide training and prediction threads. SA is sequential.

control

L (1000000), pilot (400), log_every (10000).

pairs

Variable pairs for fidelity.

conditional_on

Fidelity conditioning column, or NULL for largest reference cardinality.

keep_trace

Retain progress trace.

verbose

Print progress.

initial_indices

Optional distinct pool row indices.

initial_y

Optional numerical responses matching those rows' bins. Otherwise sampled uniformly inside the bins.

Details

The guide is trained once on categorical predictors and the original real numeric response. It is never refitted during SA. Pool predictions are cached in memory. Fidelity uses categorical marginal, selected pairwise and conditional terms, exactly as categorical landing. Utility is the absolute difference between selected-target and real-reference guide error divided by real response SD. The real reference error is evaluated on guide training rows and is not external predictive validation.

A proposal selects one target position, one currently unselected pool index and an independent uniform response in that index's bin. Integer-valued input responses use admissible integers uniformly. If all pool rows are selected, a proposal refreshes only the selected position's response. Rejection restores the complete old categorical/numeric state. The pilot evaluates and rejects proposals without modifying initial state. Only a single absolute or squared error contribution changes per step. Other numerical variables are not reconstructed here.

At temperature T, the target is proportional to exp(-H/T) relative to uniform pool-index selection and the product of within-bin uniform response laws. The proposal ratio cancels this base-measure ratio. Consequently acceptance uses exp(-delta H/T), without an interval-width correction. This is not a claim that the proposal is symmetric with respect to Lebesgue measure when bin widths differ. An energy density defined relative to Lebesgue measure instead would require a different Hastings ratio.

The final response values must be used directly, not reconstructed again. The main spades_synthesise workflow does this and reconstructs other numeric columns after selection. Arbitrary within-bin response values can adapt to the guide without preserving within-bin conditional structure: external evaluation measures predictive performance on released numerical data. Uniform random reconstruction is not a privacy guarantee. Interrupted-SA checkpoint resume is not supported by this kernel.

Value

A spades_response_landing object inheriting spades_landing, with categorical data, indices, response_values, initial_response_values, response_errors, fidelity components, energy summary, guide metadata and optional trace.

Resume or reuse a saved flight

Description

Continue the original flight from per-chain checkpoints without model refitting. Completed flights are loaded directly.

```
spades_resume_flight(checkpoint_dir, workers = 6L, verbose = TRUE,
                     checkpoint_every = 10000L, run_for = NULL)
```

Arguments

checkpoint_dir

Directory created by spades_flight with checkpoint_dir set.

workers

Number of execution workers. Logical chain count is unchanged.

verbose

Print console progress.

checkpoint_every

Checkpoint interval in MH transitions per chain.

run_for

Optional positive limit on additional transitions per chain, for a controlled pause.

Details

The original model, request and RNG states are reused. The R and dependency versions must match. An abrupt stop loses only work after the last committed checkpoint. A stale RUNNING.lock must only be removed after confirming that the previous run and its workers have stopped. See RESUME.md in the source package.

Value

A spades_flight object, or spades_flight_paused if run_for stops before completion.

Reconstruction, Fidelity, Membership Audits and Fixed-Pool Target Stability

Description

Checks original-bin reconstruction and fidelity, evaluates fixed-release membership attacks, and repeats landing on a fixed pool. These empirical checks do not establish anonymity or differential privacy.

```
spades_reconstruct_bins(categorical, discretizers,
  integer_columns = character(), seed = 4123L)
spades_fidelity_audit(reference, categorical, released, reference_values,
  discretizers = list(), pairs = list(), conditional_on = NULL)
spades_mia_candidates(members, nonmembers, seed = 20261030L)
spades_mia_stability(audit, out_dir, seeds = 20261100L + seq_len(20L),
  threads = 6L, bootstrap = 1000L, verbose = TRUE)
spades_privacy_audit(members, nonmembers, released, out_dir,
  numeric = names(members)[vapply(members, is.numeric, logical(1))],
  candidates = NULL, seeds = 20261100L + seq_len(20L),
  threads = 6L, verbose = TRUE)
spades_target_stability(pool, reference, reference_values, nonmembers,
  discretizers, integer_columns, n, out_dir, utility = NULL,
  pairs = list(), conditional_on = NULL,
  seeds = 20262000L + seq_len(10L),
  control = list(L = 1000000L, pilot = 400L, log_every = 100000L),
  mia_seeds = 20261100L + seq_len(20L), threads = 6L,
  verbose = TRUE, pool_values = NULL)
```

Arguments

categorical

Complete categorical target data frame.

discretizers

Named saved discretizers with method, levels and breaks.

integer_columns

Names of numeric variables reconstructed as integers.

seed

Local random seed, preserving caller RNG state.

reference

Complete categorical reference data frame.

released

Final reconstructed synthetic data frame, including duplicate contents.

reference_values

Original complete reference values in the same row and column order.

pairs

List of two-variable fidelity pairs.

conditional_on

Conditioning variable; defaults to highest cardinality.

members

Original records used for generation.

nonmembers

Original records excluded from generation.

audit

Saved privacy cache with features and binary labels.

out_dir

Explicit local output directory; use a short path on Windows. Files contain private candidate indices.

seeds

Distinct seeds for attacks or target replications, respectively.

threads

Threads for the membership random forests.

bootstrap

First-split stratified AUC bootstrap repetitions.

verbose

Print progress.

numeric

Numeric distance columns; other columns use equality distances.

candidates

Reusable candidate plan from spades_mia_candidates.

pool

Fixed categorical pool; no new flight is run.

n

Target size, at least five.

utility

Fixed landing utility specification or NULL.

control

Landing iteration, pilot and progress controls.

mia_seeds

Same membership split seeds for every target.

pool_values

For regression, fixed numeric pool used to compute utility scores. Selected values are reused, not reconstructed again.

Details

All frames must have matching column order and no missing or nonfinite values. Conditional fidelity is weighted by reference stratum proportions. Numeric Wasserstein distance is normalized by reference range; KS and quantile differences are also reported. Reconstruction must reproduce every original bin.

Membership attacks operate on balanced unique record profiles. Profiles shared between members and nonmembers are excluded and counted. At least 200 eligible profiles per class are required. Distances use all released rows. Numeric differences are divided by reference range and clipped at one; Hamming uses exact equality. Attacks use separate 40/30/30 percent training, calibration and test partitions, fixed score directions, and calibration-only thresholds and selection. The RF uses 500 trees and minimum node size ten. Controls include permuted training labels and a hypothetical exact-copy release. Only the first split has bootstrap intervals; overlapping split variability is descriptive. Candidate-source differences may confound membership results.

The target study preserves the fixed pool, fidelity structure and utility guide. Completed targets, features and attack splits are cached with input fingerprints. An interrupted unfinished landing restarts that target. A lock prevents concurrent target studies. After a crash a stale lock must be removed only after confirming that no process remains. Regression requires the exact fixed numeric pool underlying utility scores. No external predictive utility models are fitted by these functions.

Value

Reconstruction returns a data frame. Fidelity returns component, summary and numeric tables. Candidate selection returns row indices and labels. Privacy and stability return metric lists and write explicit audit/checkpoint files. The target study invisibly returns per-target and aggregate summaries.

External Utility and Fixed-Seed Target Stability

Description

Train external predictive models on real and released synthetic datasets, then evaluate all models on the same real holdout. This function is independent of synthesis and does not tune models on the holdout.

```
spades_external_utility(reference, synthetic, holdout, target,
  out_dir, predictors = NULL, mode = c("classification", "regression"),
  models = c("XT", "Bagging_CART", "XGB"), seed = 20261026L,
  threads = 1L, numeric = NULL, control = list(),
  missing_tokens = "NA", verbose = TRUE)
```

Arguments

reference

Complete real training data.frame.

synthetic

Complete released data.frame or named list of released data.frames, after numeric reconstruction. Dataset names must be unique and different from Real.

holdout

Complete real test data.frame, reserved from generator and predictive model training.

target

Response column name.

out_dir

Dedicated output directory. Existing results are reused only when inputs and configuration match.

predictors

Predictor names; defaults to all reference columns except the response. Exclude identifiers explicitly.

mode

Classification or regression. Classification requires every response class in every dataset, including holdout, so macro metrics are defined. Regression requires numeric responses.

models

Subset of XT, Bagging_CART and XGB. Requires ranger, rpart and xgboost respectively.

seed

One nonnegative integer, reset for every model and dataset within a scope that preserves caller RNG state. Multiple model seeds are deliberately not supported.

threads

Positive thread count for XT and XGB. CART bootstrap trees run sequentially.

numeric

Numeric predictor names, inferred from reference column types by default. Other predictors are treated as categorical, including ordered factors.

control

Named list overriding trees (500), min_node (5, at most 10), nrounds (300), eta (0.05), max_depth (4). All other model settings are fixed and recorded in protocol.rds.

missing_tokens

Text labels treated as missing, in addition to actual missing values and internal missing markers. Incomplete evaluation inputs are rejected, not silently filtered.

verbose

Print per-fit progress and compact final summary.

Details

Each training dataset has a separate encoder. Numeric predictors use training means and standard deviations; categorical predictors use training-only one-hot encoding. Previously unseen holdout categories map to all-zero indicators and their counts are recorded. Matrices are dense; high-cardinality inputs can require substantial memory.

XT uses probability forests for classification and regression forests otherwise, with extra-randomized splits, square-root feature count, no bootstrap and full sampling. Bagging CART fits bootstrap rpart trees with cp=0, minsplit=10, maxdepth=30, no cross-validation and all encoded predictors eligible. XGB uses histogram trees, subsample=0.8, colsample_bytree=0.8, min_child_weight=1, lambda=1 and alpha=0, with multiclass soft probabilities or squared-error regression. No test watchlist or early stopping is used.

The real baseline is fitted once per model and reused for comparisons with every synthetic target. Numeric reconstruction, flight and landing are never run. Target SD and ranges are descriptive and conditional on the supplied pool, holdout, model seed and synthetic datasets; reconstruction variability may contribute. Shared holdout predictions are not independent experiments. A fixed seed does not establish deterministic equivalence across hardware or library versions.

Completed evaluations are committed to individual checkpoints. An interrupted fit restarts; completed fits are reused. Configuration or input changes cause rejection of an existing output directory. Concurrent access is blocked by RUNNING.lock. After a process crash, remove a stale lock only after verifying that the run is inactive. Old standalone case-study checkpoint formats are not imported by this API. Existing standalone results remain valid and need not be rerun.

Checkpoints contain holdout responses and predictions and must be kept private. Summaries are written to metrics.csv, paired_gaps.csv and across_targets.csv. Protocol and session information are retained.

Value

Invisibly, a list with metrics, paired synthetic-minus-real gaps, descriptive across-target summary, preprocessing metadata and protocol. Classification metrics are accuracy, balanced accuracy, macro F1, log loss and the sum-over-classes Brier score. Regression metrics are RMSE, MAE and R squared (NA for constant holdout response). SD is NA for one synthetic target.